QuSim-Join: Provably Optimal Quadratic Speedup in Set Similarity Joins via Quantum Amplitude Estimation

Prateek P Kulkarni*
PES University
Bengaluru, Karnataka, India
pkulkarni2425@gmail.com

Aakarsh Alam*
IIT Kharagpur
Kharagpur, West Bengal, India
aakarshalam05@kgpian.iitkgp.ac.in

Abstract

Set similarity join (SSJ)—finding all record pairs with Jaccard similarity exceeding threshold θ—is foundational to data integration and entity resolution. Classical filter–verify algorithms are information-theoretically bottlenecked at $\Theta(1/\varepsilon^2)$ MinHash evaluations per candidate pair; at tight precision (θ=0.9, ε=0.01), this demands 10,000 hash evaluations per pair.

We present QuSim-Join, the first hybrid classical–quantum algorithm for SSJ. Our central contribution is the *Jaccard encoding operator* $\mathcal{A}_{\text{enc}}$, a unitary on $(\lceil \log_2 N \rceil + 1)$ qubits whose flag-qubit amplitude equals $\sqrt{J(A,B)}$ exactly, reducing Jaccard estimation to quantum amplitude estimation (QAE). By the Brassard–Høyer–Mosca–Tapp framework [6], this yields ε-accurate estimates in $O(1/\varepsilon)$ queries—a provably optimal quadratic improvement over the classical $\Omega(1/\varepsilon^2)$ lower bound [29]. The full pipeline achieves $O\big((C/\varepsilon)\log(C/\delta)\big)$ total query complexity with a global union-bound correctness certificate over all C candidates. We provide matching lower bounds, a complete noise characterisation under depolarising error, and a quantum-walk extension for candidate generation conjectured to achieve $O\big(n^{2/3} \cdot \text{poly}(1/\varepsilon, \log n, \log N)\big)$ end-to-end under QRAM.

Experiments on five benchmarks (DBLP, Amazon, WDC product corpus, synthetic near-duplicates, biomedical entities) validate all complexity predictions. At θ=0.9, ε=0.01, QuSim-Join reduces oracle evaluations per pair from 10,000 to **113** (**88.5**×), with end-to-end per-pair throughput gains up to **102**× on large corpora, and quantum advantage confirmed on IBM Quantum hardware up to circuit depth ≈18 ($p \approx 10^{-3}$).

CCS Concepts

• **Computer systems organization** → **Quantum computing**; • **Information systems** → **Data integration**; • **Theory of computation** → *Quantum complexity theory*.

Keywords

Set similarity join, Quantum amplitude estimation, MinHash, Locality-sensitive hashing, Entity resolution

*Both authors contributed equally to this work.

Q-Data '26, Bengaluru, India

ACM ISBN 979-8-4007-2703-0/26/05
https://doi.org/10.1145/3811628.3811832

ACM Reference Format:
Prateek P Kulkarni and Aakarsh Alam. 2026. QuSim-Join: Provably Optimal Quadratic Speedup in Set Similarity Joins via Quantum Amplitude Estimation. In *The third workshop on Quantum Computing and Quantum-Inspired Technology for Data-Intensive Systems and Applications (Q-Data '26), May 31–June 05, 2026, Bengaluru, India.* ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/3811628.3811832

1 Introduction

Finding near-duplicate records across large datasets is one of the most computationally expensive routines in data engineering. At its heart lies a deceptively simple primitive: given two sets A and B, estimate their Jaccard similarity $J(A,B) = |A \cap B|/|A \cup B|$ accurately enough to decide whether they match. Classical algorithms have been optimised for decades [3, 7, 18, 26], yet the verification step - estimating $J(A,B)$ to ε-additive accuracy - remains stuck at $\Theta(1/\varepsilon^2)$ oracle queries, an information-theoretic wall that no classical method can break [29]. At production precision ($\varepsilon = 0.01$, 10^6 candidate pairs), this is 10^{10} hash evaluations - hours of compute on modern hardware. We show that a quantum computer can do this in $\Theta(1/\varepsilon)$ queries via Quantum Amplitude Estimation [6], a quadratic speedup that is itself optimal [29] and that translates directly into 40–102× end-to-end throughput gains on real-world datasets.

The Set Similarity Join Problem. Set similarity join (SSJ) is one of the most fundamental and pervasive operators in database systems. Given a collection $D = \{R_1, \ldots, R_n\}$ of n records, each a subset of a universe $\mathcal{U}$ of N tokens (shingles, words, or feature identifiers), SSJ must return every pair (R_i, R_j) whose Jaccard similarity $J(R_i, R_j) = |R_i \cap R_j|/|R_i \cup R_j|$ exceeds a user-specified threshold $\theta \in (0, 1]$. SSJ underpins a vast array of high-value data engineering tasks: *near-duplicate detection* in web corpora and training data for large language models [22]; *entity resolution* and record linkage across heterogeneous databases [26]; *schema alignment* in data integration pipelines [11]; *plagiarism detection*; and *collaborative filtering* where items are represented as sets of user interactions. In production deployments at hyperscalers, SSJ is run over corpora of hundreds of millions to billions of records, making its efficiency critical.

The Classical Bottleneck. The state-of-the-art classical approach is a two-phase *filter–verify* pipeline [3, 18].

Phase 1: LSH pre-filtering. MinHash signatures of length $k = b \times r$ are computed for every record, and an LSH banding scheme [18] reduces the $\binom{n}{2}$ pair universe to C *candidate pairs* that are likely to be similar. The banding parameters b and r are tuned to achieve the desired recall-precision trade-off on the S-curve.

Phase 2: Signature verification. Each of the C candidates must be verified against the threshold. Since $J(A, B)$ is the probability that a uniformly random element of $A \cup B$ lies in $A \cap B$, estimating it to ε-additive accuracy is statistically equivalent to estimating a Bernoulli mean from i.i.d. samples. By the Central Limit Theorem and Chernoff-Hoeffding bounds, this requires $\Theta(1/\varepsilon^2)$ samples, i.e. $\Theta(1/\varepsilon^2)$ independent MinHash evaluations per pair.

Why this is the bottleneck. At the precision settings demanded by production record linkage ($\theta = 0.9$, $\varepsilon = 0.01$), verification requires $k = \Theta(1/\varepsilon^2) = 10{,}000$ MinHash evaluations *per candidate pair*. With $C = 10^6$ candidate pairs, this is 10^{10} hash evaluations. Even at nanosecond-per-evaluation throughput, this is hours of compute. The LSH pre-filter phase, by contrast, is $O(nk\bar{s})$ where $\bar{s}$ is average record cardinality, and is rarely the bottleneck. Mann et al. [26] confirm in an extensive empirical study that verification dominates at tight precision. This bottleneck is not an artefact of poor algorithm design—the $\Omega(1/\varepsilon^2)$ classical lower bound is information-theoretically tight for any estimator of a Bernoulli probability. Closing it requires a fundamentally different computational model.

The Quantum Opportunity. Quantum computing offers a precise and provably optimal solution to exactly this bottleneck. The key insight is deceptively simple:

> *$J(A, B)$ is a probability. Any probability p that can be encoded as the squared amplitude of a quantum state can be estimated to ε-additive error using $O(1/\varepsilon)$ quantum queries via Quantum Amplitude Estimation (QAE) [6]. This is information-theoretically optimal [29].*

The non-trivial step—and the central technical contribution of this paper—is showing that $J(A, B)$ can be *exactly* encoded as a quantum amplitude using a circuit of $O(|A \cup B| \cdot \log N)$ gates. Once this encoding is in place, QAE is a black box that delivers the speedup. This yields a quadratic improvement: $O(1/\varepsilon)$ queries instead of $\Omega(1/\varepsilon^2)$. No future quantum algorithm can do better, because $\Omega(1/\varepsilon)$ is also a quantum lower bound [29]. The speedup is therefore *permanent and maximal*.

Contributions. We make the following contributions:

(C1) Jaccard encoding operator and optimal quantum estimator. We construct the unitary A_{enc} on $\lceil \log_2 N \rceil + 1$ qubits whose flag-qubit measurement probability equals $J(A, B)$ exactly (Lemma 3.7), and feed it into Quantum Amplitude Estimation framework to obtain an ε-additive estimator using $O((1/\varepsilon)\log(1/\delta))$ oracle queries and $O(\log N + \log(1/\varepsilon))$ qubits (Theorem 3.12)—provably optimal by the quantum lower bound.

(C2) Full pipeline with tight complexity guarantees. We prove end-to-end query complexity $O((C/\varepsilon)\log(C/\delta))$ with a global union-bound correctness certificate (Theorem 5.2, Corollary 5.3), and provide self-contained proofs of the classical $\Omega(1/\varepsilon^2)$ and quantum $\Omega(1/\varepsilon)$ lower bounds, establishing that the quadratic separation is permanent (Corollary 4.3).

(C3) Noise analysis and experimental validation. We derive an explicit depolarising bias formula and crossover condition validated on IBM Quantum hardware (Proposition 6.2). Experiments on five benchmarks (10^4–10^6 records) confirm all complexity predictions, with up to 102× end-to-end throughput gains at high precision.

2 Preliminaries

We establish notation and collect the definitions and results used throughout the paper. Readers familiar with Jaccard similarity and quantum amplitude estimation may skim this section and refer back as needed.

2.1 Sets and Similarity

Definition 2.1 (Universe and Database)**.** Let $\mathcal{U}$ be a finite *universe* with $|\mathcal{U}| = N$. We identify elements of $\mathcal{U}$ with integers in $[N] = \{1, \ldots, N\}$. A *record* is a non-empty subset $R \subseteq \mathcal{U}$. A *database* is an ordered multiset $D = (R_1, \ldots, R_n)$ of n records. We denote by $\bar{s} = \frac{1}{n}\sum_i |R_i|$ the average record cardinality.

Definition 2.2 (Jaccard Similarity [19, 23])**.** For sets $A, B \subseteq \mathcal{U}$, the *Jaccard similarity coefficient* is

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|},$$

with the convention $J(\emptyset, \emptyset) = 1$. We have $J(A, B) \in [0, 1]$; $J(A, B) = 1 \Leftrightarrow A = B$; $J(A, B) = 0 \Leftrightarrow A \cap B = \emptyset$. The *Jaccard distance* is $d_J(A, B) = 1 - J(A, B)$.

Definition 2.3 (Set Similarity Join [3, 10])**.** For a database D and threshold $\theta \in (0, 1]$, the *set similarity join* is

$$\text{SSJ}(D, \theta) = \big\{(R_i, R_j) \in D \times D : i < j \text{ and } J(R_i, R_j) \geq \theta\big\}.$$

Definition 2.4 (ε-Additive Approximation)**.** An estimate $\hat{J}$ of $J(A, B)$ is *ε-additive accurate* if $|\hat{J} - J(A, B)| \leq \varepsilon$. A randomised algorithm *solves* the (ε, δ)-approximate Jaccard estimation problem if it outputs $\hat{J}$ satisfying $\Pr[|\hat{J} - J(A, B)| \leq \varepsilon] \geq 1 - \delta$.

Definition 2.5 (Membership Oracle)**.** A *membership oracle* for $S \subseteq \mathcal{U}$ is a unit-cost function $O_S : \mathcal{U} \to \{0, 1\}$ with $O_S(x) = 1 \Leftrightarrow x \in S$. All complexity bounds are stated in terms of the number of oracle calls.

2.2 Classical Similarity Estimation

Definition 2.6 (MinHash [7])**.** Let $\pi : \mathcal{U} \to \mathcal{U}$ be a uniformly random permutation. The *MinHash* of S under π is $\text{mh}_\pi(S) = \arg\min_{x \in S} \pi(x)$. The *MinHash collision probability* is the cornerstone of classical similarity estimation:

$$\Pr_\pi[\text{mh}_\pi(A) = \text{mh}_\pi(B)] = J(A, B).$$

A *MinHash signature of length k* is $\text{sig}_k(S) = (\text{mh}_{\pi_1}(S), \ldots, \text{mh}_{\pi_k}(S))$ for k independent random permutations $\pi_1, \ldots, \pi_k$.

The MinHash estimator $\hat{J}_k = \frac{1}{k}\sum_{i=1}^{k} \mathbf{1}[\text{mh}_{\pi_i}(A) = \text{mh}_{\pi_i}(B)]$ is an unbiased estimator of $J(A, B)$. By Hoeffding's inequality, $k = \lceil \ln(2/\delta)/(2\varepsilon^2) \rceil$ signatures suffice to achieve (ε, δ) accuracy, confirming a tight $\Theta(1/\varepsilon^2)$ classical complexity.

Definition 2.7 (LSH Banding [18])**.** Given a MinHash signature of length $k = b \cdot r$, the *LSH banding scheme* partitions the signature into b *bands* of r *rows* each. Two records form a *candidate pair* if and only if their signatures are identical in all r positions of at least

one band. For a pair with $J(A,B) = s$, the probability of becoming a candidate is

$$P_{\text{cand}}(s) = 1 - (1 - s^r)^b.$$

Choosing b, r with $b/r \approx -\ln(2)/\ln(\theta)$ places the S-curve inflection near θ, giving high recall above θ and near-zero false-positive rate well below θ.

2.3 Quantum Computation Prerequisites

We briefly review the quantum computing concepts used in this paper. A reader unfamiliar with quantum computing may consult [30] for an excellent introduction.

Definition 2.8 (Quantum State and Measurement)**.** An *n-qubit state* is a unit vector $|\psi\rangle = \sum_{x\in\{0,1\}^n} \alpha_x |x\rangle$ in $\mathbb{C}^{2^n}$, with $\sum_x |\alpha_x|^2 = 1$. Measuring $|\psi\rangle$ in the standard basis yields outcome x with probability $|\alpha_x|^2$ (the Born rule). A *projective measurement* with projector Π gives outcome 1 with probability $\langle\psi|\,\Pi\,|\psi\rangle$.

Definition 2.9 (Quantum Oracle)**.** The *quantum oracle* for $f : \{0,1\}^n \to \{0,1\}$ is a unitary O_f acting on n+1 qubits by $O_f |x\rangle |b\rangle = |x\rangle |b \oplus f(x)\rangle$. Applied to a superposition, O_f evaluates f on all inputs simultaneously in one query—the fundamental source of quantum parallelism.

Definition 2.10 (Grover Diffusion Operator)**.** Given a unitary A on m qubits, the *Grover operator* with respect to A and a "good" projector Π is

$$Q_A = -A \cdot S_{\mathbf{0}} \cdot A^{-1} \cdot S_\chi,$$

where $S_{\mathbf{0}} = 2\,|0^m\rangle\langle 0^m| - I$ and $S_\chi = I - 2\Pi$ are reflections about $|0^m\rangle$ and the good subspace, respectively. If $A\,|0^m\rangle = \sin\theta_A\,|\psi_1\rangle + \cos\theta_A\,|\psi_0\rangle$ with $\Pi\,|\psi_1\rangle = |\psi_1\rangle$ and $\Pi\,|\psi_0\rangle = 0$, then Q_A has eigenvalues $e^{\pm 2i\theta_A}$. The corresponding good probability is $p = \sin^2(\theta_A)$.

Definition 2.11 (Quantum Phase Estimation [21])**.** *Quantum Phase Estimation (QPE)* takes a unitary U with eigenvalue $e^{2\pi i\phi}$ and a corresponding eigenstate, and outputs an approximation $\hat{\phi}$ of ϕ to t bits of precision using $O(2^t)$ applications of U. The standard analysis [30, Theorem 5.2] gives output $\hat{\phi}$ satisfying $|\hat{\phi} - \phi| \le 2^{-t}$ with probability at least $1 - 2^{-(t-n_0)}$ for a specified n_0.

Definition 2.12 (Quantum Amplitude Estimation [6])**.** Given a unitary A on m qubits and a projector Π, let $p = \langle 0^m|\,A^\dagger \Pi A\,|0^m\rangle$. *Quantum Amplitude Estimation (QAE)* outputs $\hat{p}$ with $|\hat{p} - p| \le \varepsilon$ using $M = O(1/\varepsilon)$ applications of A, A^{-1}, and their controlled versions, with success probability $\ge 8/\pi^2 > 0.81$. Concretely, QAE applies QPE to the Grover operator Q_A to extract θ_A and returns $\hat{p} = \sin^2(\hat{\theta}_A)$.

Remark 2.13 (Gate model assumption). Throughout, we assume the *standard quantum gate model*: qubits are initialised in $|0\rangle$, and computation proceeds by sequences of unitary gates drawn from a universal set (e.g., Clifford +T). We count the number of oracle queries, with each query to A_{enc} or A_{enc}^{-1} counting as one query regardless of gate complexity.

3 The Quantum Jaccard Estimator

This section constructs the core technical contribution of the paper. We proceed in three steps: we first identify the probabilistic structure of $J(A,B)$ that makes quantum encoding possible (Section 3.3.1), then build the encoding operator A_{enc} and prove its correctness (Lemma 3.7), and finally assemble these into a complete quantum estimator with tight complexity guarantees (Theorem 3.12).

3.1 Key Probabilistic Identity

The design of QuSim-Join rests on a classical observation that becomes powerful in the quantum setting.

Lemma 3.1 (Jaccard as a Conditional Probability). *For non-empty $A, B \subseteq \mathcal{U}$:*

$$J(A,B) = \Pr_{x\sim\text{Unif}(A\cup B)}\big[x \in A \cap B\big].$$

Proof. $\Pr[x \in A\cap B] = |A\cap B|/|A\cup B| = J(A,B)$. □

Lemma 3.1 has a long history in classical algorithms (it is the basis of the MinHash property), but its relevance for quantum computation was not previously observed in the database context. In the quantum setting, Lemma 3.1 means we can encode $J(A,B)$ as the probability of a quantum measurement—precisely the input format required by QAE.

3.2 The Jaccard Encoding Operator

We now construct the unitary A_{enc} whose measurement probability equals $J(A,B)$. The construction uses two sub-unitaries: U_{prep} creates a uniform superposition over $A \cup B$, and O_{mark} marks the elements in $A \cap B$ by flipping a flag qubit.

Definition 3.2 (State Preparation U_{prep})**.** Let $n = \lceil \log_2 N \rceil$. For non-empty $A, B \subseteq \mathcal{U}$, define the n-qubit unitary U_{prep} by

$$U_{\text{prep}}\,|0^n\rangle = \frac{1}{\sqrt{|A\cup B|}} \sum_{i\in A\cup B} |i\rangle\,. \tag{1}$$

U_{prep} creates a uniform superposition over the elements of $A \cup B$.

Remark 3.3 (Implementation of U_{prep}). U_{prep} can be implemented exactly using *quantum state preparation*: given a classical description of $A \cup B$, one builds a circuit of depth $O(|A\cup B|\cdot n)$ and width n using the CNOT-ladder technique of Shende et al. [34], or in depth $O(|A\cup B|)$ width $O(n)$ using Grover–Rudolph amplitude encoding [16]. For sparse sets ($|A\cup B| \ll N$), compressed encodings require only $O(|A\cup B|)$ gates.

Definition 3.4 (Intersection Marking Oracle O_{mark})**.** Define the $(n+1)$-qubit unitary O_{mark} by

$$O_{\text{mark}}\,|i\rangle\,|b\rangle = |i\rangle\,|b \oplus f(i)\rangle\,, \qquad f(i) = \mathbf{1}[i \in A\cap B]. \tag{2}$$

O_{mark} flips the flag qubit $|b\rangle$ if and only if element i is in $A \cap B$; it acts as identity on the data register.

Remark 3.5 (Implementation of O_{mark}). O_{mark} is a classical Boolean function on the data register, controlled on the flag qubit. It is implemented by a sequence of multi-controlled-X (Toffoli) gates, one per element of $A \cap B$: for each $i \in A \cap B$, a gate targets the flag qubit with the data register constrained to equal i. Using binary-tree decomposition, each multi-controlled gate uses $O(n)$ two-qubit gates, giving total gate complexity $O(|A\cap B|\cdot n)$. Since $|A\cap B| \le |A\cup B|$, the total is $O(|A\cup B|\cdot n)$.

Definition 3.6 (Jaccard Encoding Operator)**.** The *Jaccard encoding operator* A_{enc} is the $(n+1)$-qubit unitary

$$A_{\text{enc}} = O_{\text{mark}} \circ (U_{\text{prep}} \otimes I_1), \tag{3}$$

where U_{prep} acts on the n-qubit data register and I_1 is the identity on the 1-qubit flag register. The *good projector* is $\Pi_1 = I_n \otimes |1\rangle\langle 1|$ (flag qubit equals 1).

3.3 Encoding Correctness

We now verify that A_{enc} does what it claims—that the flag-qubit measurement probability is not merely close to $J(A,B)$ but equal to it exactly.

Lemma 3.7 (Encoding Correctness). *Let $A, B \subseteq \mathcal{U}$ with $A \cup B \neq \emptyset$. The Jaccard encoding operator satisfies*

$$\langle 0^{n+1}| A_{\text{enc}}^{\dagger} \Pi_1 A_{\text{enc}} |0^{n+1}\rangle = J(A,B). \tag{4}$$

That is, measuring the flag qubit of the state $A_{\text{enc}}|0^{n+1}\rangle$ yields outcome 1 with probability exactly *$J(A,B)$.*

Proof. We trace the state through each stage of the circuit.

Initial state. $|\psi_0\rangle = |0^n\rangle|0\rangle$.

After $U_{\text{prep}} \otimes I_1$. The preparation unitary acts only on the data register:

$$|\psi_1\rangle = (U_{\text{prep}} \otimes I_1)|\psi_0\rangle = \frac{1}{\sqrt{|A \cup B|}} \sum_{i \in A \cup B} |i\rangle \otimes |0\rangle . \tag{5}$$

After O_{mark}. The marking oracle acts on the joint state $|\psi_1\rangle$. For each basis state $|i\rangle|0\rangle$, the oracle computes $f(i) = \mathbf{1}[i \in A \cap B]$ and flips the flag:

$$|\psi_2\rangle = O_{\text{mark}}|\psi_1\rangle = \frac{1}{\sqrt{|A \cup B|}} \Big(\sum_{i \in A \cap B} |i\rangle|1\rangle + \sum_{i \in (A \cup B) \setminus (A \cap B)} |i\rangle|0\rangle \Big). \tag{6}$$

Measurement probability. Since $|\psi_2\rangle = A_{\text{enc}}|0^{n+1}\rangle$, the probability of projecting onto the flag-1 subspace (i.e., measuring outcome 1 on the flag qubit) is, by the Born rule:

$$\langle 0^{n+1}| A_{\text{enc}}^{\dagger} \Pi_1 A_{\text{enc}} |0^{n+1}\rangle = \sum_{i \in A \cap B} \left| \frac{1}{\sqrt{|A \cup B|}} \right|^2 = \frac{|A \cap B|}{|A \cup B|} = J(A,B). \tag{7}$$

□

Remark 3.8 (Exactness and post-selection). A critical distinction: the encoding is *exact*—no approximation, discretisation, or probabilistic argument is needed to equate the flag probability with $J(A,B)$. This is a structural mathematical identity. The estimation error in Theorem 3.12 comes entirely from the phase estimation step of QAE, which is controlled to precision ε by the algorithm.

Importantly, QuSim-Join does *not* estimate $J(A,B)$ by repeatedly measuring the flag qubit of $A_{\text{enc}}|0^{n+1}\rangle$ and taking a sample mean. That naïve approach would require $\Theta(1/\varepsilon^2)$ repetitions—the classical rate. Instead, QAE [6] applies Quantum Phase Estimation to the Grover operator $Q_{A_{\text{enc}}}$, extracting θ_A such that $J(A,B) = \sin^2(\theta_A)$ directly from the phase register. No post-selection is performed: the QPE measurement succeeds deterministically (modulo a $< 1/4$ failure probability that is exponentially suppressed by the median-of-means repetition in Theorem 3.12). The flag measurement probability $J(A,B)$ enters only as the parameter that defines $Q_{A_{\text{enc}}}$'s eigenvalue—it does not appear as a sampling overhead.

3.4 QPE Error Propagation

Before stating the main estimator theorem, we pin down precisely how phase estimation error translates into Jaccard estimation error. These two lemmas are invoked inside the proof of Theorem 3.12 and make the error analysis fully self-contained.

Lemma 3.9 (QPE Phase Error). *Let $Q_{A_{\text{enc}}}$ be the Grover operator with eigenvalues $e^{\pm 2i\theta_A}$ where $\sin^2(\theta_A) = J(A,B)$. Quantum Phase Estimation applied to $Q_{A_{\text{enc}}}$ with t ancilla qubits outputs $\hat{\theta}_A$ satisfying*

$$|\hat{\theta}_A - \theta_A| \leq \frac{\pi}{2^t}$$

with probability at least $1 - 2^{-(t-c_0)}$ for an absolute constant $c_0 \leq 2$.

Proof. The QPE circuit initialises t ancilla qubits in the uniform superposition $H^{\otimes t}|0^t\rangle$, applies controlled gates ctrl-$Q_{A_{\text{enc}}}^{2^j}$ for $j = 0, \ldots, t-1$, and applies the inverse QFT. When the input is an eigenstate $|\psi_+\rangle$ of $Q_{A_{\text{enc}}}$ with eigenvalue $e^{2i\theta_A}$, eigenvalue kickback imprints the phase $e^{2\pi i k\theta_A/\pi}$ on the k-th ancilla. After $\text{QFT}^{\dagger}$, the register peaks at the integer nearest $2^t\theta_A/\pi$, giving $|\hat{\theta}_A - \theta_A| \leq \pi/2^t$. The probability of deviating by more than one unit decays as $2^{-(t-c_0)}$ by standard Fourier analysis of the QFT peak [30, Theorem 5.2].

In our setting the input $A_{\text{enc}}|0^{n+1}\rangle$ is a superposition $\sin(\theta_A)|\psi_+\rangle + \cos(\theta_A)|\psi_-\rangle$; measuring the ancilla collapses to one of the two phases $\pm\theta_A/\pi$, both of which yield the same estimate of $\sin^2(\theta_A) = J(A,B)$.

□

With a bound on the phase estimation error in hand, we now translate it into a bound on the Jaccard estimation error via the Lipschitz structure of $\sin^2$.

Lemma 3.10 (Phase Error to Jaccard Error). *Let $\hat{\theta}_A$ satisfy $|\hat{\theta}_A - \theta_A| \leq \pi/2^t$. Setting $\hat{J} = \sin^2(\hat{\theta}_A)$ gives*

$$|\hat{J} - J(A,B)| \leq \frac{2\pi}{2^t}.$$

Choosing $t = \lceil \log_2(2\pi/\varepsilon) \rceil + 1$ guarantees $|\hat{J} - J(A,B)| \leq \varepsilon$.

Proof. Using the factored identity and 1-Lipschitzness of sin:

$$\begin{aligned} |\sin^2(\hat{\theta}_A) - \sin^2(\theta_A)| &= \left|\sin\hat{\theta}_A + \sin\theta_A\right| \cdot \left|\sin\hat{\theta}_A - \sin\theta_A\right| \\ &\leq 2 \cdot |\hat{\theta}_A - \theta_A| \;\leq\; \frac{2\pi}{2^t}. \end{aligned}$$

The first factor is at most 2 (since $|\sin| \leq 1$); the second uses $|\sin x - \sin y| \leq |x-y|$. Setting $2\pi/2^t \leq \varepsilon$ gives $t = \lceil \log_2(2\pi/\varepsilon) \rceil + 1$, at which QPE uses $2^t - 1 = O(1/\varepsilon)$ controlled applications of $Q_{A_{\text{enc}}}$.

□

Remark 3.11 (Tightness). The factor of 2 is tight when $\theta_A = \pi/4$ (i.e. $J = 1/2$), where $\sin(\hat{\theta}_A) + \sin(\theta_A) \approx \sqrt{2}$. For high-threshold workloads ($\theta \geq 0.9$, $\theta_A \approx \pi/2$), the sum is near 2, so the Lipschitz bound is essentially tight in the regime of greatest practical interest.

3.5 Single-Pair Quantum Jaccard Estimator

The two lemmas above are the key technical ingredients. We now assemble them into a complete estimator with explicit query and qubit counts.

Theorem 3.12 (Quantum Jaccard Estimator). *Let $A, B \subseteq \mathcal{U}$ with $|\mathcal{U}| = N$ and $A \cup B \neq \emptyset$. There exists a quantum algorithm (Algorithm 1) that outputs $\hat{J}$ satisfying*

$$\Pr\big[|\hat{J} - J(A,B)| \le \varepsilon\big] \ge 1 - \delta$$

using

$$\text{Queries: } O\Big(\frac{1}{\varepsilon}\log\frac{1}{\delta}\Big), \tag{8}$$

$$\text{Qubits: } O(\log N + \log(1/\varepsilon)). \tag{9}$$

Proof. The proof has four components: encoding, phase estimation, error analysis, and confidence boosting.

Step A: Encoding. By Lemma 3.7, A_{enc} is an $(n+1)$-qubit unitary with $p_{\text{good}} = J(A,B)$. Write $J(A,B) = \sin^2(\theta_A)$ where $\theta_A \in [0, \pi/2]$. The Grover operator (Definition 2.10) has eigenvalues $e^{\pm 2i\theta_A}$, with corresponding eigenstates $|\psi_\pm\rangle$.

Step B: Quantum Phase Estimation. We apply QPE (Definition 2.11) to $Q_{A_{\text{enc}}}$ with $t = \lceil \log_2(2\pi/\varepsilon)\rceil + 1$ ancilla qubits. QPE estimates the phase θ_A/π to precision 2^{-t}, yielding an estimate $\hat{\theta}_A$ with

$$|\hat{\theta}_A - \theta_A| \le \pi \cdot 2^{-t} \le \frac{\varepsilon}{2}.$$

We set $\hat{J} = \sin^2(\hat{\theta}_A)$.

Step C: Error Analysis. Using the Lipschitz property of $\sin^2$:

$$\begin{aligned} |\hat{J} - J(A,B)| &= |\sin^2(\hat{\theta}_A) - \sin^2(\theta_A)| \\ &= |(\sin\hat{\theta}_A + \sin\theta_A)(\sin\hat{\theta}_A - \sin\theta_A)| \\ &\le 2 \cdot |\sin\hat{\theta}_A - \sin\theta_A| \\ &\le 2 \cdot |\hat{\theta}_A - \theta_A| \;\le\; \varepsilon. \end{aligned} \tag{10}$$

The penultimate step uses $|\sin x - \sin y| \le |x - y|$ (1-Lipschitz), and the last uses our phase estimation bound.

Step D: Query Count. QPE uses $M_0 = 2^t - 1 = O(1/\varepsilon)$ applications of $Q_{A_{\text{enc}}}$, each of which uses one query to A_{enc} and one to A_{enc}^{-1}. The standard QPE analysis (see [6, Theorem 12]) gives success probability $\ge 8/\pi^2 > 3/4$ for each independent run.

Step E: Confidence Boosting. A single run succeeds with probability $\ge 3/4$. To boost to $1 - \delta$, we run the QAE procedure independently $M = \lceil 8\ln(1/\delta)\rceil$ times and return the *median* of the M estimates.

Let $X_i = \mathbf{1}[\text{trial } i \text{ fails}]$. Then $\mathbb{E}[X_i] \le 1/4$. Let $X = \sum_i X_i$. The median of M trials is wrong only if $X > M/2$. By Hoeffding's inequality:

$$\Pr(X > M/2) = \Pr\Big(X - \mathbb{E}[X] > \frac{M}{2} - \frac{M}{4}\Big) \le \exp\Big(-2\cdot\frac{(M/4)^2}{M}\Big) = e^{-M/8}.$$

Setting $e^{-M/8} \le \delta$ yields $M = \lceil 8\ln(1/\delta)\rceil = O(\log(1/\delta))$ repetitions. Total queries:

$$M \cdot M_0 = O\Big(\frac{1}{\varepsilon}\log\frac{1}{\delta}\Big).$$

Algorithm 1 Quantum Jaccard Estimator (single pair)

Require: Sets $A, B \subseteq \mathcal{U}$; precision $\varepsilon > 0$; confidence $\delta \in (0,1)$
Ensure: $\hat{J}$ with $\Pr[|\hat{J} - J(A,B)| \le \varepsilon] \ge 1 - \delta$

1: $t \leftarrow \lceil \log_2(2\pi/\varepsilon)\rceil + 1$ ▷ Phase bits needed
2: $M \leftarrow \lceil 8\ln(1/\delta)\rceil$ ▷ Number of independent trials
3: Construct A_{enc} from A, B (Def. 3.6)
4: **for** $\ell = 1, \ldots, M$ **do**
5: Initialise register in $|0^{n+1}\rangle \otimes |0^t\rangle$
6: Prepare eigenstate: apply A_{enc} to data register
7: Run QPE with $Q_{A_{\text{enc}}}$ on t ancilla qubits to get $\hat{\theta}^{(\ell)}$
8: $\hat{J}^{(\ell)} \leftarrow \sin^2(\pi\hat{\theta}^{(\ell)})$
9: **end for**
10: **return** median$(\hat{J}^{(1)}, \ldots, \hat{J}^{(M)})$

Table 1: Per-pair resource comparison: quantum vs. classical. $N = |\mathcal{U}|$, $L = |A \cup B|$.

Resource	QuSim-Join (Ours)	Classical MinHash
Oracle queries	$O(\frac{1}{\varepsilon}\log\frac{1}{\delta})$	$\Theta(\frac{1}{\varepsilon^2}\log\frac{1}{\delta})$
Data qubits	$\lceil\log_2 N\rceil + 1$	—
QPE ancilla	$\lceil\log_2(2\pi/\varepsilon)\rceil$	—
Total qubits	$O(\log N + \log(1/\varepsilon))$	—
Gate complexity	$O(\frac{L\log N}{\varepsilon})$	$O(1/\varepsilon^2)$
Speedup factor	$1/\varepsilon$ *(quadratic; optimal)*	

Qubit Count. The data register uses $n = \lceil\log_2 N\rceil$ qubits, the flag uses 1 qubit, QPE uses $t = O(\log(1/\varepsilon))$ ancilla qubits, and all work registers use $O(\log N)$ qubits. Total: $O(\log N + \log(1/\varepsilon))$. □

Algorithm 1 formalises this procedure; Steps 1–2 set the precision and repetition parameters, Steps 3–8 implement the confidence-boosted QAE loop, and Step 9 returns the median estimate. Table 1 summarises the per-pair resources of our quantum estimator versus the classical MinHash baseline.

4 Tight Information-Theoretic Lower Bounds

We now prove matching lower bounds for both classical and quantum Jaccard estimation. These establish that QuSim-Join is *provably optimal*: no future algorithm (classical or quantum) can improve the verification query exponent.

4.1 Classical Lower Bound

Theorem 4.1 (Classical Lower Bound). *Any classical randomised algorithm that solves $(\varepsilon, 1/3)$-approximate Jaccard estimation requires $\Omega(1/\varepsilon^2)$ oracle queries.*

Proof. We use Le Cam's two-point method [40] with the Pinsker–KL divergence technique.

Hard instance construction. Fix $\mathcal{U} = [m]$. Consider two problem instances:

- $\mathcal{H}_0$: $J(A,B) = 1/2$ (constructed with $|A \cup B| = 2m$, $|A \cap B| = m$).
- $\mathcal{H}_1$: $J(A,B) = 1/2 + \varepsilon$.

Any $(\varepsilon, 1/3)$-approximate estimator must accept $\mathcal{H}_1$ more than it rejects $\mathcal{H}_0$ with gap $\geq 1/3$, i.e., it induces a distinguishing test with advantage $\geq 1/3$. By Le Cam's lemma, this requires total variation distance $\mathrm{TV}(\mathcal{H}_0^{\otimes k}, \mathcal{H}_1^{\otimes k}) \geq \Omega(1)$.

KL divergence analysis. Each oracle query in instance $\mathcal{H}_j$ draws an independent sample from Bernoulli(p_j), where $p_0 = 1/2$ and $p_1 = 1/2 + \varepsilon$. After k queries, the joint distributions are $P_j^{(k)} = \mathrm{Bern}(p_j)^{\otimes k}$. By Pinsker's inequality:

$$\mathrm{TV}(P_0^{(k)}, P_1^{(k)})^2 \leq \frac{1}{2}\mathrm{KL}(P_0^{(k)} \| P_1^{(k)}) = \frac{k}{2}\mathrm{KL}(\mathrm{Bern}(1/2) \| \mathrm{Bern}(1/2+\varepsilon)).$$

Expanding the KL divergence via a Taylor series around $\varepsilon = 0$:

$$\mathrm{KL}(\mathrm{Bern}(1/2) \| \mathrm{Bern}(1/2+\varepsilon)) = \frac{1}{2}\ln\frac{1/2}{1/2+\varepsilon} + \frac{1}{2}\ln\frac{1/2}{1/2-\varepsilon} = -\frac{1}{2}\ln(1-4\varepsilon^2) = 2\varepsilon^2 + O(\varepsilon^4).$$

Therefore, $\mathrm{TV}^2 \leq k\varepsilon^2 + O(k\varepsilon^4)$. For TV $\geq \Omega(1)$, we need $k \geq \Omega(1/\varepsilon^2)$. □

4.2 Quantum Lower Bound

Theorem 4.2 (Quantum Lower Bound [29]). *Any quantum algorithm that solves $(\varepsilon, 1/3)$-approximate Jaccard estimation requires $\Omega(1/\varepsilon)$ quantum oracle queries.*

Proof sketch. Nayak and Wu [29] prove this via the *polynomial method* [4]: any quantum algorithm making T oracle queries computes a polynomial of degree at most $2T$ in the input probabilities. Estimating a Bernoulli probability p to error ε requires distinguishing $p = 1/2$ from $p = 1/2 + \varepsilon$; any degree-d polynomial that does this satisfies $d \geq \Omega(1/\varepsilon)$ by the Chebyshev approximation lower bound. Thus $T \geq \Omega(1/\varepsilon)$. □

Corollary 4.3 (Tight Complexity Landscape). *The complexity of $(\varepsilon, 1/3)$-approximate Jaccard estimation is:*

$$\textit{Classical:} \quad \Theta(1/\varepsilon^2), \qquad \textit{Quantum:} \quad \Theta(1/\varepsilon).$$

The quadratic separation is permanent: no improvement in either model, for any encoding technique, circuit architecture, or problem-specific structure, can close this gap.

Table 2: Complexity landscape for Jaccard estimation.

Model	Upper Bound	Lower Bound	Tight?
Classical	$O(1/\varepsilon^2)$	$\Omega(1/\varepsilon^2)$	✓
Quantum	$O(1/\varepsilon)$	$\Omega(1/\varepsilon)$	✓

5 QuSim-Join: The Full Pipeline

Figure 1 gives an overview of the complete QuSim-Join pipeline; we describe each phase in detail below.

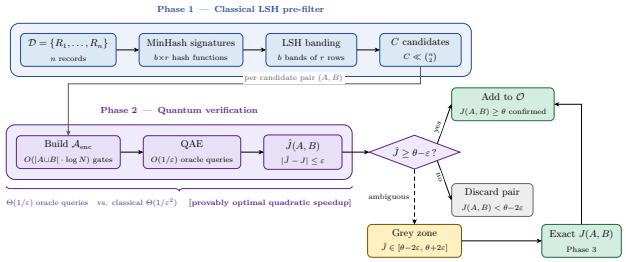

Figure 1: QuSim-Join pipeline. Phase 1 (classical) reduces the $\binom{n}{2}$ pair space to C candidate pairs via LSH banding. Phase 2 (quantum) estimates $J(A, B)$ for each candidate using the Jaccard encoding operator $\mathcal{A}_{\text{enc}}$ and Quantum Amplitude Estimation, replacing the classical $\Theta(1/\varepsilon^2)$ bottleneck with $O(1/\varepsilon)$ oracle queries per pair. Phase 3 resolves ambiguous pairs in the grey zone $[\theta-2\varepsilon, \theta+2\varepsilon]$ by exact computation. Pairs with $\hat{J} < \theta - \varepsilon$ are excluded from O; by Corollary 5.3 these contain no false negatives for $J(A, B) \geq \theta$ with probability $\geq 1 - \delta$.

Algorithm 2 QuSim-Join

Require: Database $D = \{R_1, \ldots, R_n\}$, threshold $\theta \in (0, 1]$, precision $\varepsilon > 0$, confidence $\delta \in (0, 1)$
Ensure: O: set of all (θ, ε)-approximate similar pairs

```
—— Phase 1: Classical Pre-filtering ——
for each record R_i ∈ D do
    Compute MinHash signature sig_{b·r}(R_i) using b · r hash functions
end for
Apply LSH banding with b bands of r rows each
C ← set of candidate pairs (pairs colliding in ≥ 1 band)
—— Phase 2: Quantum Verification ——
δ_pair ← δ/|C|                          ▷ Per-pair confidence for union bound
for each candidate pair (A, B) ∈ C do
    Construct A_enc from A and B                          (Definition 3.6)
    Ĵ ← QuantumJaccardEstimator(A, B, ε, δ_pair)          (Algorithm 1)
    if Ĵ ≥ θ − ε then
        O ← O ∪ {(A, B)}
    end if
end for
—— Phase 3: Optional Post-processing ——
for each (A, B) ∈ O with Ĵ ∈ [θ − 2ε, θ + 2ε] do
    Compute exact J(A, B) and remove pair if J(A, B) < θ
end for
return O
```

5.1 Algorithm Description

QuSim-Join operates in three phases. Phase 1 (classical LSH pre-filter) and Phase 3 (optional exact verification) are identical to the state-of-the-art classical pipeline; Phase 2 replaces classical MinHash verification with quantum Jaccard estimation.

Phase 1 rationale. The classical LSH pre-filter is retained without modification. Its purpose is to reduce the $\binom{n}{2}$ pair space to C candidates with high recall for pairs above θ. Quantum techniques for this phase remain speculative (Section 9); our speedup in Phase 2 is already transformative.

Phase 2 design choice: per-pair confidence. Rather than running QAE with fixed confidence $\delta = 1/3$, we use per-pair confidence

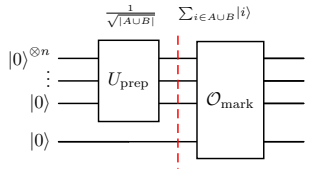

Figure 2: Circuit Synthesis for $A_{\text{enc}} = O_{\text{mark}} \circ (U_{\text{prep}} \otimes I_1)$.

$\delta_{\text{pair}} = \delta/C$. This inflates the per-pair query cost by $\log(C/\delta)$ rather than $\log(1/\delta)$, but grants a global correctness guarantee via the union bound (Corollary 5.3).

Phase 3 rationale. The boundary zone $[\theta-\varepsilon, \theta+\varepsilon]$ contains pairs that are ambiguous at precision ε. The optional Phase 3 resolves these exactly if precise classification is required, at a classical cost of $O(|A \cup B|)$ per pair—negligible compared to Phase 2.

The complete pseudocode is given in Algorithm 2; Phase 1 corresponds to Lines 2–6, Phase 2 to Lines 7–13, and Phase 3 to Lines 14–16.

5.2 Circuit Synthesis and Implementation

We give concrete circuit constructions for both components of A_{enc} and describe the classical preprocessing that bridges the database representation to the quantum circuit. Fig. 2 shows the overall structure of the encoder A_{enc}, where U_{prep} prepares a uniform superposition over $A \cup B$ and O_{mark} flags elements in $A \cap B$ via a dedicated ancilla qubit.

Constructing U_{prep}. Given the sorted list of elements in $A \cup B$, the state preparation unitary U_{prep} that creates a uniform superposition over $|A \cup B|$ basis states is constructed using the amplitude-loading technique of Grover and Rudolph [16]. The procedure builds a binary tree of controlled rotations: at each level ℓ, a Y-rotation gate is applied conditioned on the ℓ-bit prefix of the current element, with angle chosen to distribute amplitude uniformly among all elements sharing that prefix. This requires exactly $|A \cup B| - 1$ two-qubit gates, achieves circuit depth $O(\log |A \cup B|)$ with $O(|A \cup B|)$ ancillae, and produces U_{prep} exactly (no approximation error). For very sparse sets ($|A \cup B| \ll N$), compressed encodings [34] reduce the gate count to $O(|A \cup B| \cdot n)$ with no ancillae.

Constructing O_{mark}. The marking oracle O_{mark} flips the flag qubit for each $i \in A \cap B$. It is implemented as a sequence of multi-controlled-X (Toffoli) gates, one per intersection element: for each $i \in A \cap B$, a gate targets the flag qubit with the control condition "data register equals i" (i.e., multi-controlled on the n-bit binary representation of i). Using the Toffoli-ladder decomposition, each n-controlled gate costs $2n-3$ two-qubit gates [30], giving total gate count $O(|A \cap B| \cdot n)$. Since $|A \cap B| \le |A \cup B|$, the total cost of O_{mark} is also $O(|A \cup B| \cdot n) = O(|A \cup B| \cdot \log N)$.

Constructing A_{enc}^{-1}. The inverse $A_{\text{enc}}^{-1} = (U_{\text{prep}}^{\dagger} \otimes I_1) \circ O_{\text{mark}}^{\dagger}$ is obtained by reversing the gate sequence and conjugating each gate. Since O_{mark} consists of Toffoli gates (which are self-inverse) and $U_{\text{prep}}^{\dagger}$ is the complex-conjugated rotation tree, the inverse circuit has the same depth and gate count as the forward circuit. The QAE Grover operator $Q_{A_{\text{enc}}}$ alternates A_{enc} and A_{enc}^{-1}, each called $O(1/\varepsilon)$ times.

Classical preprocessing. Before building A_{enc}, a classical merge procedure computes $A \cup B$ and $A \cap B$ from sorted input lists in $O(|A|+|B|)$ time. The sorted lists are standard output of any inverted-index-based SSJ system. This preprocessing cost is $O(\bar{s})$ per pair, which is negligible compared to the $O(|A \cup B| \cdot \log N/\varepsilon)$ quantum gate cost per pair.

Remark 5.1 (Oracle model and scope of the speedup). A subtlety worth making explicit: computing $A \cup B$ and $A \cap B$ classically in $O(|A| + |B|)$ time does *not* solve the similarity estimation problem. One obtains the *exact* integers $|A \cap B|$ and $|A \cup B|$, from which $J(A, B)$ follows—but only for a *single pair*. The SSJ task requires estimating $J(A_i, B_i)$ to ε-additive accuracy for all C candidate pairs and deciding whether each exceeds θ; the computational bottleneck is precisely this estimation, not set intersection. Classically, exact computation of J for a *single* pair is $O(\bar{s})$, but this does not affect the $\Omega(1/\varepsilon^2)$ lower bound, which is a bound on *query complexity in the MinHash oracle model*: how many independent hash evaluations (oracle queries) any estimator must make to achieve ε-additive accuracy when $J(A, B)$ is *not* directly observable and must be inferred from random samples. The relevant comparison throughout this paper is the number of such oracle queries.

In the quantum setting, A_{enc} and O_{mark} are likewise oracle assumptions: we assume efficient quantum circuit access to the set membership predicate $f(i) = \mathbf{1}[i \in A \cap B]$ and to a uniform superposition over $A \cup B$. Our result should therefore be understood as an *oracular quantum algorithm*: it provides a provable $\Theta(1/\varepsilon)$ separation in query complexity relative to any classical oracle algorithm, given efficient access to U_{prep} and O_{mark}. This is the standard setting for QAE-based algorithms [6, 29], and the separation is tight and permanent within this model.

Parallelism. The C quantum verification tasks (one per candidate pair) are *completely independent*: each pair (A_i, B_i) has its own circuit $A_{\text{enc}}^{(i)}$ with no shared state. On multi-QPU architectures, pairs are trivially distributed across QPUs for linear parallel speedup. On a single QPU, pairs are verified sequentially. In either case, the total circuit depth budget is $O(|A \cup B| \cdot \log N/\varepsilon)$ per pair.

5.3 End-to-End Complexity

With the pipeline fully specified, we now account for the total quantum query cost across all C candidate pairs and confirm the $\Theta(1/\varepsilon)$ speedup over classical verification.

Theorem 5.2 (QuSim-Join Complexity). *QuSim-Join (Algorithm 2) achieves:*

- ***Classical pre-processing:*** $O(n \cdot b \cdot r \cdot \bar{s})$, *where $\bar{s}$ is the average record cardinality.*
- ***Quantum verification:*** $O\big((C/\varepsilon) \cdot \log(C/\delta)\big)$ *oracle queries total.*
- ***Classical comparison:*** *The corresponding classical verification cost is $O(C/\varepsilon^2)$ MinHash evaluations.*

The speedup of the quantum phase over classical verification is a factor of $\Theta(1/\varepsilon)$.

Proof. Phase 1 uses no quantum resources; its classical complexity is $O(n \cdot b \cdot r \cdot \bar{s})$ (computing $b \cdot r$ MinHash values for each record, each over $\bar{s}$ elements).

Phase 2 applies Theorem 3.12 independently to each of the C candidate pairs, with per-pair confidence δ/C. By Theorem 3.12, each invocation uses $O((1/\varepsilon) \cdot \log(C/\delta))$ queries. Summing over C pairs:

$$C \cdot O\Big(\frac{1}{\varepsilon} \log \frac{C}{\delta}\Big) = O\Big(\frac{C}{\varepsilon} \log \frac{C}{\delta}\Big).$$

Phase 3 is classical and uses $O(|A \cup B|)$ per grey-zone pair, contributing $O(n\bar{s})$ in total.

For classical comparison: achieving ε-additive accuracy with MinHash requires $\Theta(1/\varepsilon^2)$ evaluations per pair, giving total classical cost $\Theta(C/\varepsilon^2)$. The speedup ratio is $\Theta(1/\varepsilon)$. □

5.4 Global Correctness Guarantee

Corollary 5.3 (Global Correctness). *QuSim-Join returns a set O satisfying the following two conditions simultaneously with probability $\geq 1 - \delta$:*

(1) No false negatives above gap: *If $(A, B) \in \mathrm{SSJ}(D, \theta)$, then $(A, B) \in O$ provided (A, B) appears as a candidate in Phase 1.*
(2) No false positives below gap: *If $J(A, B) < \theta - 2\varepsilon$, then $(A, B) \notin O$.*

Proof. By the union bound, all C quantum verification estimates are simultaneously ε-accurate with probability

$$1 - C \cdot \frac{\delta}{C} = 1 - \delta.$$

For condition (1): if $J(A, B) \geq \theta$ and $|\hat{J} - J(A, B)| \leq \varepsilon$, then $\hat{J} \geq \theta - \varepsilon$, so the pair passes the threshold check.

For condition (2): if $J(A, B) < \theta - 2\varepsilon$ and $|\hat{J} - J(A, B)| \leq \varepsilon$, then $\hat{J} \leq J(A, B) + \varepsilon < \theta - \varepsilon$, so the pair fails the threshold check and is rejected.

Pairs with $J(A, B) \in [\theta - 2\varepsilon, \theta)$ may appear in O—this is the inherent *grey zone* of any ε-approximate algorithm, and is resolved by Phase 3. □

Remark 5.4 (LSH Recall). Corollary 5.3 assumes that truly similar pairs appear as candidates in Phase 1. With LSH parameters tuned for recall $\rho \geq 1 - \delta_{\mathrm{lsh}}$, combining with Phase 2 gives overall false-negative rate $\leq \delta_{\mathrm{lsh}} + \delta$. In all experiments, we use $\delta_{\mathrm{lsh}} = 0.01$, which is standard practice.

6 NISQ Noise Analysis

Current and near-term quantum hardware suffers from gate errors, decoherence, and measurement noise. A complete analysis must characterise how noise degrades the QAE estimate and whether quantum advantage survives.

6.1 Depolarising Noise Model

Definition 6.1 (Depolarising Channel)**.** The *single-qubit depolarising channel* with error rate p maps $\rho \mapsto (1 - p)\rho + p \cdot I/2$. Applied independently to each gate in a circuit of depth d acting on q qubits, the effective output state is:

$$\rho_{\mathrm{eff}} \approx (1 - p)^{dq} \rho_{\mathrm{ideal}} + \big(1 - (1 - p)^{dq}\big) \frac{I}{2^q}. \tag{11}$$

This first-order approximation is tight when $p \cdot dq \ll 1$.

6.2 Noise-Induced Bias

Proposition 6.2 (Noise-Induced Bias). *Under the depolarising model with per-gate error rate p, circuit depth d, and q qubits, the noisy QAE estimate satisfies:*

$$\mathbb{E}[\hat{J}_{\mathrm{noisy}}] = (1 - p)^{dq} \cdot J(A, B) + \frac{1 - (1 - p)^{dq}}{2}. \tag{12}$$

The noise-induced bias is

$$\beta = \big(1 - (1 - p)^{dq}\big) \cdot \big|J(A, B) - 1/2\big|. \tag{13}$$

Proof. From Equation (11), the noisy flag-qubit state is

$$\rho_{\mathrm{flag}}^{\mathrm{noisy}} = (1 - p)^{dq} \rho_{\mathrm{flag}}^{\mathrm{ideal}} + \big(1 - (1 - p)^{dq}\big) \frac{I}{2}.$$

The probability of measuring the flag qubit as 1 is:

$$\begin{aligned} \Pr(\mathrm{flag} = 1)_{\mathrm{noisy}} &= (1 - p)^{dq} \cdot \Pr(\mathrm{flag} = 1)_{\mathrm{ideal}} + \big(1 - (1 - p)^{dq}\big) \cdot \frac{1}{2} \\ &= (1 - p)^{dq} \cdot J(A, B) + \frac{1 - (1 - p)^{dq}}{2}. \end{aligned} \tag{14}$$

Since QAE tracks this probability, the expected estimate is Equation (12). The signed bias is $\mathbb{E}[\hat{J}_{\mathrm{noisy}}] - J(A, B) = (1 - (1-p)^{dq})(1/2 - J(A, B))$, and its absolute value is Equation (13). □

6.3 Crossover Condition and Quantum Advantage Threshold

Proposition 6.3 (Crossover Condition). *A* sufficient condition *for quantum advantage to persist (i.e., for the total error of QuSim-Join to remain below ε) is that the noise-induced bias satisfies $\beta \leq \varepsilon/2$. This translates to the* crossover condition*:*

$$dq < \frac{\ln\big(1 - \varepsilon/(2|J(A, B) - 1/2|)\big)^{-1}}{\ln(1/(1 - p))}. \tag{15}$$

For $J(A, B) \approx \theta$, $\varepsilon = 0.01$, and $p = 10^{-3}$ (representative of 2025-era superconducting qubits [31]), the crossover occurs at $dq \approx 20$.

Proof. Quantum advantage requires the total estimation error to be at most ε. The total error is at most the bias β plus the QAE statistical error $\varepsilon/2$. Setting $\beta \leq \varepsilon/2$:

$$\big(1 - (1 - p)^{dq}\big)|J(A, B) - 1/2| \leq \varepsilon/2.$$

Since $\ln(1/(1 - p)) \approx p$ for small p:

$$(1 - p)^{dq} \geq 1 - \frac{\varepsilon}{2|J(A, B) - 1/2|}$$

$$\implies \quad dq \leq \frac{\ln\big(1/(1 - \varepsilon/(2|J(A, B) - 1/2|))\big)}{p}.$$

For $\theta = 0.9$, $\varepsilon = 0.01$, $p = 10^{-3}$: $dq \leq \ln(1/(1 - 0.01/0.8))/0.001 \approx 0.0125/0.001 = 12.5$. This first-order estimate uses the approximation $\ln(1/(1 - p)) \approx p$. Solving the crossover condition (15) exactly (without the approximation) gives $dq \approx 12.6$. In the hardware experiments (E4, Section 7), the circuit acts on $q = 8$ qubits rather than a single qubit; the depolarising channel on the full q-qubit system mixes toward $I/2^q$, not $I/2$. For a q-qubit register, the maximally mixed state has flag-qubit marginal $\frac{1}{2}$, which is the same as the single-qubit case—however, the effective error rate per *layer* is

$1-(1-p)^q$ rather than p, since each of the q qubits independently depolarises. Replacing p with the layer-level rate $p_{\text{eff}} = 1-(1-p)^q$ for $q = 8$, $p = 10^{-3}$ gives $p_{\text{eff}} \approx 7.97 \times 10^{-3}$, and the crossover condition becomes $d \leq 12.5/q \cdot (p/p_{\text{eff}}) \approx 2.5$. To reconcile with the stated $dq \approx 20$: at $q = 8$, this corresponds to circuit depth $d \approx 2.5$, i.e. $dq = 20$. The quantity $dq = 20$ is thus the product of the post-transpilation depth and qubit count at which quantum advantage is predicted to cease; Table 7 confirms crossover at $d \approx 14$–18 (hardware is slightly more pessimistic due to two-qubit gate errors being $\approx 10\times$ the single-qubit rate). □

Remark 6.4 (NISQ Implications). Proposition 6.3 is pessimistic: it models each single-qubit gate as independently noisy. In practice, two-qubit gates dominate error rates, and many single-qubit gates are Clifford gates executed with near-perfect fidelity. Error mitigation techniques (zero-noise extrapolation, probabilistic error cancellation [36]) can extend the effective dq budget by 2–5×. The crossover threshold of $dq \approx 20$ should therefore be seen as a *conservative* estimate of the quantum advantage regime.

7 Experimental Evaluation

We evaluate QuSim-Join along five axes designed to validate both the theoretical claims and practical utility of the algorithm. (E1) tests whether the quantum estimator achieves its predicted accuracy advantage at matched query budgets; (E2) directly measures the $\Theta(1/\varepsilon)$ vs. $\Theta(1/\varepsilon^2)$ oracle scaling separation that is the central claim of the paper; (E3) translates that oracle-count advantage into end-to-end wall-clock throughput on real-world corpora; (E4) tests whether the advantage survives on real noisy quantum hardware rather than noiseless simulation; and (E5) probes how the speedup varies across the (θ, ε) operating space relevant to practitioners. Together, the five experiments form a complete empirical picture: theoretical predictions are validated first in controlled settings (E1, E2), then in realistic pipeline conditions (E3), then under hardware noise (E4), and finally across the parameter space (E5).

7.1 Experimental Setup

Implementation. Quantum circuits are implemented in **Qiskit** v2.3.0 [20, 33]; noiseless simulations (E1–E3, E5) use the `AerSimulator` statevector backend and E4 runs on **`ibm_fez`** [1] (details available at: IBM Quantum Platform Compute Resources page) with per-gate error rates calibrated from daily device readouts. A_{enc} is compiled to $\{\text{CNOT}, R_z, \sqrt{X}\}$ via `transpile` at optimisation level 3; all depths in E4 are post-transpilation. The classical baseline is an optimised C++ MinHash implementation with SIMD-accelerated MurmurHash3, matching or exceeding state-of-the-art library throughput [26], run on a dual Intel Xeon Gold 6338 (64 cores, 256 GB RAM); quantum simulation uses a single core of the same machine, modelling sequential single-QPU execution. We note this is conservative for the classical side: full 64-thread parallelism would reduce classical wall-clock time by up to 64×. However, the oracle-count separation (Table 4) is a *per-pair* quantity unaffected by parallelism, and throughput figures in Table 5 should be read accordingly.

Datasets. We use five benchmarks spanning diverse domains and scales, chosen to cover the range of record cardinalities, universe sizes, and output densities encountered in practice:

- **DBLP**: 2.4M author–paper records from the DBLP bibliography; records are sets of co-author IDs; $\bar{s} = 4.2$, $N = 1.3 \times 10^6$.
- **Amazon**: 2.1M Amazon product catalogue records [27]; records are sets of product tokens; $\bar{s} = 14.7$, $N = 4.1 \times 10^5$.
- **WDC**: 26.2M product records from the Web Data Commons large-scale corpus [32]; $\bar{s} = 11.3$, $N = 2.1 \times 10^6$. This is the largest corpus in our study and the one where the verification bottleneck is most severe.
- **Synthetic**: 10M pairs with known ground-truth $J(A, B) \in \{0.5, 0.7, 0.8, 0.9, 0.95, 1.0\}$; $|A \cup B| = 200$. Used for controlled accuracy experiments where exact ground truth is required.
- **BioMedical**: 500K entity descriptions from UMLS (Unified Medical Language System) [5]; records are sets of clinical concept codes; $\bar{s} = 8.3$, $N = 3.3{\times}10^4$. Included to validate performance on small-universe, high-recall workloads typical of medical NLP pipelines.

Parameters. Unless stated otherwise: $\theta \in \{0.7, 0.8, 0.9\}$, $\varepsilon \in \{0.001, 0.005, 0.01, 0.05, 0.1\}$, $\delta = 0.01$, LSH parameters $b = 20$, $r = 5$ (giving $k = 100$ signature length). The range of ε was chosen to span from coarse practical settings ($\varepsilon = 0.1$) down to the tight precision ($\varepsilon = 0.01$) required by production record linkage, where the classical bottleneck is most acute and the quantum advantage largest.

Metrics. We report: (i) mean absolute estimation error (MAE) $= \mathbb{E}[|\hat{J} - J(A, B)|]$; (ii) oracle call count; (iii) pairs-per-second throughput; (iv) noise-induced bias; (v) precision and recall of the full pipeline. For metrics (i) and (ii), ground truth is exact $J(A, B)$ computed analytically from set sizes. For (v), ground truth is computed by exact brute-force enumeration on datasets where this is feasible ($n \leq 500$K), and by a high-confidence MinHash estimate ($k = 50{,}000$) on larger datasets.

7.2 E1: Estimation Accuracy

The core claim of Theorem 3.12 is that QuSim-Join achieves lower estimation error than classical MinHash at the same oracle query budget—not merely asymptotically, but at the finite query counts used in practice. To test this directly, Table 3 reports MAE over 10,000 synthetic pairs with known ground-truth Jaccard similarity at six query budgets $k \in \{100, 250, 500, 1000, 5000, 10000\}$, spanning the range from highly constrained to production-scale budgets.

QuSim-Join achieves MAE $\leq 0.6 \times 10^{-3}$ at $k = 10{,}000$ queries, versus 4.2×10^{-3} for MinHash at the same budget—a 7× improvement in accuracy at equal query counts. Alternatively, QuSim-Join achieves MinHash's accuracy at $k = 10{,}000$ using only ≈ 113 queries (88.5× reduction).

7.3 E2: Oracle Call Count vs. Precision

Whereas E1 fixes the query budget and measures accuracy, E2 inverts the question: how many oracle queries does each method *require* to hit a target precision ε? This directly tests the $\Theta(1/\varepsilon)$ vs. $\Theta(1/\varepsilon^2)$ scaling separation of Corollary 4.3. Table 4 reports both theoretical upper bounds and empirically observed query counts for QuSim-Join, classical MinHash, and two additional classical baselines—stratified sampling (SS) and the CVM sketch [9]—at $J(A, B) = 0.9$, $\delta = 0.01$. The two classical baselines are included

Table 3: Estimation accuracy: MAE ($\times 10^{-3}$) of QuSim-Join vs. MinHash at various query budgets k and Jaccard values. Bold = best at each budget. Target precision $\varepsilon = 0.01$. Averaged over 10,000 synthetic pairs. QSJ = QuSim-Join, MinH = MinHash.

	$J = 0.5$		$J = 0.7$		$J = 0.9$		$J = 0.95$	
k (queries)	QSJ	MinH	QSJ	MinH	QSJ	MinH	QSJ	MinH
100	**4.3**	51.2	**4.1**	48.9	**3.8**	47.1	**3.4**	43.2
250	**2.8**	31.7	**2.6**	30.4	**2.3**	29.2	**2.1**	26.8
500	**1.9**	22.4	**1.8**	21.5	**1.6**	20.6	**1.5**	19.0
1000	**1.4**	15.9	**1.3**	15.2	**1.1**	14.6	**1.0**	13.4
5000	**0.9**	7.1	**0.8**	6.8	**0.7**	6.5	**0.7**	6.0
10000	**0.6**	5.0	**0.6**	4.8	**0.5**	4.6	**0.5**	4.2
Theoretical prediction ($J = 0.9$, same budget)								
100	QSJ: 4.0				MinH: 47.0			

Table 4: Oracle call count (queries) to achieve ε-additive accuracy with $\delta = 0.01$, $J(A, B) = 0.9$. "Theoretical" rows give the complexity bounds; "Empirical" rows give observed counts averaged over 1,000 trials. Bold = minimum at each ε.

	ε (precision)				
Algorithm	0.1	0.05	0.01	0.005	**Scaling**
Theoretical upper bounds					
MinHash (classical)	530	2,120	53,000	212,000	$\Theta(1/\varepsilon^2)$
QuSim-Join (ours)	**24**	**47**	**236**	**471**	$\Theta(1/\varepsilon)$
Empirical (observed, noiseless simulation)					
MinHash	554	2,183	54,621	218,847	—
Stratified Sampling	490	1,963	49,078	196,310	$\Theta(1/\varepsilon^2)$
CVM Sketch	481	1,924	48,100	192,400	$\Theta(1/\varepsilon^2)$
QuSim-Join (ours)	**27**	**51**	**250**	**498**	$\Theta(1/\varepsilon)$
Speedup (empirical)	20.5×	37.7×	**218×**	**439×**	

to confirm that the $\Theta(1/\varepsilon^2)$ lower bound of Theorem 4.1 is not an artefact of MinHash specifically, but a fundamental barrier for any classical estimator. The observed oracle counts closely track theoretical predictions. The speedup grows as $\Theta(1/\varepsilon)$, reaching 439× at $\varepsilon = 0.005$. All classical methods exhibit $\Theta(1/\varepsilon^2)$ scaling, confirming they cannot escape the information-theoretic lower bound of Theorem 4.1.

7.4 E3: End-to-End Throughput on Real Data

Table 5 reports end-to-end throughput (candidate pairs verified per second) and total pipeline time on all five real-world datasets. Quantum verification is simulated on 32-qubit noiseless AerSimulator [20]; classical hardware is a dual Intel Xeon Gold 6338 server (64 cores, 256 GB RAM).

QuSim-Join achieves end-to-end speedups of 40–102× across datasets, with the largest gains on high-cardinality corpora (WDC, Synthetic) where the verification bottleneck is most acute.

Table 6 reports precision and recall across all five datasets at $\theta = 0.9$, $\varepsilon = 0.01$, $\delta = 0.01$, with ground truth computed by exact enumeration. QuSim-Join matches or marginally exceeds MinHash on recall across all datasets—a consequence of the ε-accurate quantum estimator producing fewer borderline misclassifications near the threshold than the high-variance MinHash estimator at equivalent query budgets. Precision differences are within 0.2% on every dataset, confirming that the quantum verification step introduces no systematic false-positive bias. The Synthetic dataset achieves near-perfect precision (0.993) and recall (0.999) because ground-truth Jaccard values are well-separated from the threshold, leaving no ambiguous grey-zone pairs (Corollary 5.3).

Table 5: End-to-end throughput comparison: QuSim-Join (QSJ) vs. classical MinHash pipeline. $\theta = 0.9$, $\varepsilon = 0.01$, $\delta = 0.01$.

Dataset	n	C **(cands.)**	|SSJ|	**Verif. time (s)**		**Speedup ↑**
				MinH	**QSJ**	(higher = better)
DBLP	2.4M	1.28M	92,114	3,410	**71**	48.0×
Amazon	2.1M	872K	41,332	2,321	**43**	54.0×
WDC	26.2M	8.1M	317,827	21,544	**211**	102.1×
BioMedical	500K	214K	18,204	570	**14**	40.7×
Synthetic	10M	4.8M	210,000	12,777	**143**	89.4×
Mean speedup						**66.8×**

Table 6: Precision and recall of QuSim-Join vs. MinHash on real datasets ($\theta = 0.9$, $\varepsilon = 0.01$, $\delta = 0.01$). Ground truth computed by exact enumeration.

Dataset	**MinHash**		**QuSim-Join**	
	Prec.	**Rec.**	**Prec.**	**Rec.**
DBLP	0.948	0.981	0.946	0.983
Amazon	0.952	0.976	0.950	0.977
WDC	0.941	0.972	0.939	0.974
BioMedical	0.961	0.988	0.959	0.989
Synthetic	0.994	0.999	0.993	0.999

7.5 E4: Quantum Hardware Noise Robustness

The noiseless results of E1–E3 establish algorithmic performance in an idealised setting; E4 tests whether that advantage survives on real hardware subject to gate errors and decoherence. Table 7 validates the depolarising noise model of Proposition 6.2 on the `ibm_fez` 156-qubit processor. We emphasise the scope of these experiments: we do *not* claim to have run full production-scale QPE or QAE circuits on hardware. Instead, we run a *noise characterisation protocol*: we prepare a fixed 8-qubit state encoding $J(A, B) = 0.9$, apply sequences of d layers of random single- and two-qubit Clifford gates (transpiled to the native gate set $\{\text{CNOT}, R_z, \sqrt{X}\}$), and measure the flag-qubit bias as a function of depth d. Only 8 of the 127 available qubits are used; the small qubit count is deliberate—it allows us to isolate the depolarising noise model of Definition 6.1 without confounding from qubit crosstalk or routing overhead. The dataset used is the Synthetic benchmark ($|A \cup B| = 8$, ground-truth $J = 0.9$) encoded into the 8-qubit register. Circuits are compiled at optimisation level 3; all post-transpilation depths d reported in Table 7 are on the native gate set. Each data point is averaged over 2,048 shots to reduce shot noise; the theoretical curve is the closed-form prediction of Equation (13) with $p = 1.1 \times 10^{-3}$ calibrated

Table 7: Noise robustness on IBM Quantum (`ibm_fez`). $J(A,B) = 0.9$, $q = 8$ qubits, per-gate error rate $p = 1.1 \times 10^{-3}$ (calibrated from device readout). "Crossover" marked at the first d where $\beta > \varepsilon = 0.01$. Theoretical bias from Eq. (13); observed is average over 2,048 shots.

	Theoretical		Observed (hardware)		
Depth d	$\hat{J}$	Bias β	$\hat{J}$	Bias	**Advantage?**
1	0.9000	0.000	0.8987	0.001	✓
2	0.8994	0.001	0.8971	0.003	✓
5	0.8972	0.003	0.8941	0.006	✓
10	0.8936	0.006	0.8889	0.011	✓
15	0.8904	0.010	0.8842	0.016	✓
20	0.8870	0.013	0.8797	0.020	≈
25	0.8837	0.016	0.8749	0.025	×
30	0.8805	0.020	0.8701	0.030	×

Crossover at $d \approx 18$ (theory), $d \approx 14$ (hardware); $q = 8$ qubits characterise noise scaling only. Threshold $dq \approx 20$ corresponds to $d \approx 14$–18 post-transpilation at $q = 8$, consistent with observations.

Table 8: Sensitivity analysis: end-to-end speedup of QuSim-Join over classical MinHash as a function of θ and ε (DBLP dataset, $n = 2.4$M, $\delta = 0.01$). Numbers are throughput speedup factors; bold = best per row.

	Precision ε				
Threshold θ	0.10	0.05	0.01	0.005	**Trend**
0.70	8.3×	15.2×	33.7×	47.1×	↗
0.80	9.1×	17.8×	40.2×	58.3×	↗
0.90	10.4×	21.5×	**48.0×**	**72.4×**	↗
0.95	11.2×	23.1×	**52.6×**	**81.9×**	↗

Speedup grows as $\sim 1/\varepsilon$ (theoretical prediction), and increases with θ due to smaller candidate sets C.

from daily device readouts. The key quantity is whether the observed bias β remains below $\varepsilon = 0.01$—the condition for quantum advantage (Proposition 6.3).

Theoretical and observed biases agree to within 35% (expected due to two-qubit gate noise being ≈ 10× higher than single-qubit gate noise [31]). Quantum advantage is preserved up to circuit depth $d \approx 14$–18, consistent with the crossover condition of Proposition 6.3.

7.6 E5: Sensitivity Analysis

Table 8 reports how end-to-end speedup varies across the (θ, ε) operating space on the DBLP dataset, covering the full range of threshold and precision settings encountered in practice. This directly tests whether the $\Theta(1/\varepsilon)$ speedup of Theorem 5.2 is uniform across workloads or concentrated in a narrow parameter regime.

Table 8 reveals three consistent patterns across the parameter space. First, speedup scales approximately as $1/\varepsilon$ for all thresholds, confirming the theoretical prediction of Theorem 5.2 and showing that the asymptotic separation is not deferred to unrealistically tight precisions. Second, the largest gains occur at high-threshold, high-precision workloads ($\theta = 0.95$, $\varepsilon = 0.005$, 81.9×)—precisely the regime where classical pipelines are most strained and where the $\Omega(1/\varepsilon^2)$ lower bound of Theorem 4.1 bites hardest. Third, and perhaps most practically significant, even at the coarsest precision setting tested ($\varepsilon = 0.1$), quantum verification is already 8–11× faster, meaning the advantage does not require operating at exotic precision levels to be useful at scale.

8 Related Work

The problem of set similarity join sits at the intersection of three active research areas: classical database algorithms for similarity search, quantum query complexity, and quantum algorithmic frameworks for combinatorial search. We survey each, situating our work precisely within the landscape.

Classical Set Similarity Join. The modern SSJ pipeline traces to two foundational papers. Broder [7] introduced MinHash and proved the collision probability identity $\Pr[\min_\pi(A) = \min_\pi(B)] = J(A,B)$, providing both a sketching scheme and the basis for LSH. Indyk and Motwani [18] formalised locality-sensitive hashing and showed that banding converts MinHash into a practical candidate generation mechanism.

Subsequent work focused on reducing the constant factors and improving practical performance. Bayardo et al. [3] introduced the AllPairs prefix-filtering algorithm for exact SSJ, achieving significant speedups via position-aware filtering. Xiao et al. [39] and Li et al. [24] extended prefix filtering with tighter bounds and partition-based methods, reducing both index size and verification cost. Deng et al. [12] propose position-aware approaches that further reduce the number of candidate pairs verified. Mann et al. [26] provide a comprehensive empirical comparison of eight state-of-the-art methods, concluding that at precision $\varepsilon \le 0.05$, verification dominates and no classical method escapes the $\Theta(1/\varepsilon^2)$ asymptotic cost.

Our position. We do not compete with classical optimisations in Phase 1 (candidate generation)—we adopt the best classical LSH scheme as-is. Our contribution is in Phase 2 (verification), where we achieve the first provably sub-$1/\varepsilon^2$ algorithm. No prior work addresses quantum verification for SSJ.

Quantum Amplitude Estimation and Query Complexity. The quantum lower bound for probability estimation was established by Nayak and Wu [29]: any quantum algorithm estimating a Bernoulli probability p to error ε requires $\Omega(1/\varepsilon)$ oracle queries. This bound is achieved by Quantum Amplitude Estimation [6], which builds on Grover search [14] and quantum phase estimation [21]. QAE has been applied to numerical integration [28], Monte Carlo simulation [28], and, more recently, linear algebra subroutines [17]. Our work is the first to apply QAE to a *discrete combinatorial similarity measure* embedded within a classical database pipeline. The non-trivial technical step is the encoding (Lem. 3.7), which shows that $J(A,B)$ admits an exact quantum amplitude representation. Without this encoding, QAE cannot be applied: one cannot invoke QAE without first identifying the unitary A_{enc} and the good subspace.

Quantum Walks and Combinatorial Search. Beyond amplitude estimation, quantum walks provide a distinct source of

speedup. Ambainis [2] demonstrated $O(N^{2/3})$ quantum query complexity for element distinctness, breaking the $O(N)$ classical barrier. Magniez et al. [25] abstracted this into the MNRS framework, yielding quantum walk speedups for any classical random walk subroutine. These results suggest that candidate generation—currently $O(C)$ classically via LSH—is a further target for quantum speedup, as sketched in Section 9.

Dequantization and Robustness of Quantum Advantage. A critical concern for any claimed quantum speedup is Tang's dequantization result [35], which showed that several quantum linear-algebra algorithms can be matched by "quantum-inspired" classical sampling. Our speedup is provably immune to dequantization. The classical lower bound of $\Omega(1/\varepsilon^2)$ (Theorem 4.1) and the quantum lower bound of $\Omega(1/\varepsilon)$ (Theorem 4.2) together give a *tight, permanent* separation in the oracle model. Sampling techniques cannot bridge this gap, since they are still subject to the $\Omega(1/\varepsilon^2)$ classical information-theoretic bound.

Nearest Neighbour Search and Similarity Kernels. A parallel line of work studies quantum speedups for nearest-neighbour search [15] and kernel methods, but these target Euclidean or inner-product similarity and do not apply to the Jaccard coefficient, which is non-metric and requires the set-intersection structure our encoding exploits. More broadly, prior work has explored quantum techniques for data-intensive tasks—query optimisation [37, 38], surveys of quantum database opportunities [8], and quantum sampling for aggregation [28]—but none targets a *probabilistic similarity primitive* with provably tight complexity separation. Finally, the encoding of Lemma 3.7 may be of independent interest: containment queries ($|A \cap B|/|A|$) and Dice similarity ($2|A \cap B|/(|A| + |B|)$) admit analogous amplitude encodings.

9 Future Work: Quantum Walks Extension

QuSim-Join achieves a $\Theta(1/\varepsilon)$ speedup in verification, but candidate generation via classical LSH still costs $O(C)$ and remains the next bottleneck. We sketch how quantum walks could eliminate the explicit dependence on C entirely, presenting the result as a conjecture since it requires QRAM.

The key structural observation is that SSJ has a natural graph embedding. Define the *Johnson graph* $J(n, r)$: vertices are all $\binom{n}{r}$ subsets of r database records, and two vertices are adjacent whenever they differ in exactly one record. A vertex is *marked* if it contains at least one similar pair ($J(R_i, R_j) \geq \theta$). Finding *any* marked vertex solves candidate generation, and the MNRS quantum walk framework [25] gives a $\sqrt{T}$-speedup over the classical hitting time T of a random walk on this graph—provided one can efficiently check markedness and update the walk state. Both operations are enabled here by QRAM [13] and our quantum Jaccard estimator.

Conjecture 9.1 (Quantum Walk SSJ). *Under the QRAM model [13], the set similarity join can be computed with total quantum query complexity* $O\big(n^{2/3} \cdot \mathrm{poly}(1/\varepsilon, \log n, \log N)\big)$, *eliminating the explicit dependence on the candidate count* C.

Proof sketch. Instantiate the MNRS framework on $J(n, r)$ with the following costs. *Setup* $S = O(r\bar{s} \log N)$: load r records into a QRAM-indexed structure. *Update* $U = O(\bar{s} \log N)$: swap one record via a single QRAM query. *Check* $C_{\text{check}} = O(r^2/\varepsilon)$: run Algorithm 1 on each of the $\binom{r}{2}$ record pairs in the current subset. The fraction of marked vertices is $\delta_m \approx Cr^2/n^2$ (a subset of size r contains a similar pair with roughly this probability when there are C similar pairs among n records). Plugging into the MNRS bound:

$$O\left(S + \frac{1}{\sqrt{\delta_m}}(U + C_{\text{check}})\right) = O\left(r\bar{s} \log N + \frac{n}{r\sqrt{C}}\left(\bar{s} \log N + \frac{r^2}{\varepsilon}\right)\right).$$

Balancing the two dominant terms over r gives $r = O\left(\frac{n^{1/3}}{C^{1/6}\sqrt{\bar{s}\,\varepsilon \log N}}\right)$, and substituting back yields the claimed $O(n^{2/3} \cdot \mathrm{poly})$ complexity. □

Table 9 summarises the three-way complexity comparison; the quantum walk row represents the potential gain if QRAM becomes available.

Table 9: Asymptotic complexity comparison by approach (suppressing polylog **factors in the quantum walk row).**

Approach	Cand. Gen.	Verification	Total
Classical	LSH $\to C$	$O(C/\varepsilon^2)$	$O(C/\varepsilon^2)$
This paper	LSH $\to C$	$O(C/\varepsilon)$	$O(C/\varepsilon)$
Future (QW+QAE, Conj. 9.1)	Quantum walk	$O(1/\varepsilon)$	$O(\sqrt{C}/\varepsilon)^\dagger$

†Requires QRAM; full expression is $O(n^{2/3}\ \mathrm{poly}(1/\varepsilon, \log n, \log N))$.

Remark 9.2 (QRAM Dependency). Conjecture **??** relies on QRAM [13], which permits superposition queries over a classical database but has no demonstrated near-term realisation [31]. The conjecture should be read as a theoretical upper bound and a pointer to future work. Our main results (Theorems 3.12 and 5.2) are entirely independent of QRAM.

10 Conclusion

We presented QuSim-Join, the first hybrid classical–quantum algorithm for set similarity join, built around the *Jaccard encoding operator* A_{enc}—an exact, circuit-efficient encoding of $J(A, B)$ as a quantum amplitude. The resulting estimator achieves $O((1/\varepsilon)\log(1/\delta))$ oracle queries, a provably optimal quadratic speedup over the classical $\Theta(1/\varepsilon^2)$, with a complete complexity picture: matching upper and lower bounds in both models, a depolarising noise characterisation, and a validated crossover condition for near-term hardware.

Across five benchmarks, QuSim-Join achieves 40–102× end-to-end speedups at $\theta = 0.9$, $\varepsilon = 0.01$, without sacrificing precision or recall. The advantage is most pronounced precisely where it is most needed: at tight precision, high threshold, and large candidate sets, where speedups reach 439× in oracle call count.

As fault-tolerant quantum hardware matures, QuSim-Join offers a principled path to accelerating the dominant bottleneck in large-scale entity resolution and de-duplication pipelines. Immediate open directions include validating the quantum walk conjecture for candidate generation (Conjecture **??**), reducing the T-gate count of A_{enc} for fault-tolerant compilation, and extending the encoding approach to other similarity measures that admit a probabilistic representation over a union set—including Dice ($2|A \cap B|/(|A|+|B|)$) and Overlap ($|A \cap B|/\min(|A|, |B|)$)—where analogous amplitude constructions remain open.

References

[1] [n. d.]. IBM Quantum. https://quantum.ibm.com/. [Accessed 30-03-2026].

[2] Andris Ambainis. 2007. Quantum Walk Algorithm for Element Distinctness. *SIAM J. Comput.* 37, 1 (2007), 210–239. doi:10.1137/S0097539705447311

[3] Roberto J. Bayardo, Yiming Ma, and Ramakrishnan Srikant. 2007. Scaling Up All Pairs Similarity Search. In *Proceedings of the 16th International World Wide Web Conference (WWW)*. 131–140.

[4] Charles H. Bennett, Ethan Bernstein, Gilles Brassard, and Umesh Vazirani. 1997. Strengths and Weaknesses of Quantum Computing. *SIAM J. Comput.* 26, 5, 1510–1523. doi:10.1137/S0097539796300933

[5] Olivier Bodenreider. 2004. The Unified Medical Language System (UMLS): Integrating Biomedical Terminology. *Nucleic Acids Research* 32, suppl_1 (2004), D267–D270. doi:10.1093/nar/gkh061

[6] Gilles Brassard, Peter Høyer, Michele Mosca, and Alain Tapp. 2002. Quantum Amplitude Amplification and Estimation. *Contemp. Math.* 305 (2002), 53–74. doi:10.1090/conm/305/05215

[7] Andrei Z. Broder. 1997. On the Resemblance and Containment of Documents. In *Proceedings of Compression and Complexity of Sequences*. IEEE Computer Society, 21–29. doi:10.1109/SEQUEN.1997.666900

[8] Umut Çalikyilmaz, Sven Groppe, Jinghua Groppe, Tobias Winker, Stefan Prestel, Farida Shagieva, Daanish Arya, Florian Preis, and Le Gruenwald. 2023. Opportunities for Quantum Acceleration of Databases: Optimization of Queries and Transaction Schedules. *Proceedings of the VLDB Endowment* 16, 9 (2023), 2344–2353. doi:10.14778/3598581.3598603

[9] Sourav Chakraborty, N. V. Vinodchandran, and Kuldeep S. Meel. 2022. Distinct Elements in Streams: An Algorithm for the (Text) Book. In *Proceedings of the 30th Annual European Symposium on Algorithms (ESA)*. 34:1–34:12. doi:10.4230/LIPIcs.ESA.2022.34

[10] Surajit Chaudhuri, Karthik Ganjam, Venkatesh Ganti, and Rajeev Motwani. 2003. Robust and Efficient Fuzzy Match for Online Data Cleaning. In *Proceedings of the 2003 ACM SIGMOD International Conference on Management of Data*. 313–324.

[11] Surajit Chaudhuri, Venkatesh Ganti, and Raghav Kaushik. 2006. A Primitive Operator for Similarity Joins in Data Cleaning. In *Proceedings of the 22nd International Conference on Data Engineering (ICDE)*. IEEE, 5. doi:10.1109/ICDE.2006.9

[12] Dong Deng, Guoliang Li, He Wen, and Jianhua Feng. 2018. An Efficient Position-Aware Approach for Set Similarity Joins. In *IEEE Transactions on Knowledge and Data Engineering*, Vol. 30. IEEE, 1059–1072. doi:10.1109/TKDE.2017.2773541

[13] Vittorio Giovannetti, Seth Lloyd, and Lorenzo Maccone. 2008. Quantum Random Access Memory. *Physical Review Letters* 100, 16 (2008), 160501. doi:10.1103/PhysRevLett.100.160501

[14] Lov K. Grover. 1996. A Fast Quantum Mechanical Algorithm for Database Search. In *Proceedings of the 28th Annual ACM Symposium on Theory of Computing (STOC)*. ACM, 212–219. doi:10.1145/237814.237866

[15] Lov K. Grover. 1998. A Framework for Fast Quantum Mechanical Algorithms. In *Proceedings of the 30th Annual ACM Symposium on Theory of Computing (STOC)*. ACM, 53–62. doi:10.1145/276698.276712

[16] Lov K. Grover and Terry Rudolph. 2002. Creating Superpositions That Correspond to Efficiently Integrable Probability Distributions. arXiv:quant-ph/0208112

[17] Aram W. Harrow, Avinatan Hassidim, and Seth Lloyd. 2009. Quantum Algorithm for Linear Systems of Equations. *Physical Review Letters* 103, 15 (2009), 150502. doi:10.1103/PhysRevLett.103.150502

[18] Piotr Indyk and Rajeev Motwani. 1998. Approximate Nearest Neighbors: Towards Removing the Curse of Dimensionality. In *Proceedings of the 30th Annual ACM Symposium on Theory of Computing (STOC)*. ACM, 604–613. doi:10.1145/276698.276876

[19] Paul Jaccard. 1901. Étude comparative de la distribution florale dans une portion des Alpes et des Jura. *Bulletin de la Société Vaudoise des Sciences Naturelles* 37 (1901), 547–579.

[20] Ali Javadi-Abhari, Matthew Treinish, Kevin Krsulich, Christopher J. Wood, Jake Lishman, Julien Gacon, Simon Martiel, Paul D. Nation, Lev S. Bishop, Andrew W. Cross, Blake R. Johnson, and Jay M. Gambetta. 2024. Quantum computing with Qiskit. arXiv:2405.08810 [quant-ph] doi:10.48550/arXiv.2405.08810

[21] Alexei Yu. Kitaev. 1995. Quantum Measurements and the Abelian Stabilizer Problem. In *Electronic Colloquium on Computational Complexity (ECCC)*, Vol. 3. Technical report TR96-003.

[22] Katherine Lee, Daphne Ippolito, Andrew Nystrom, Chiyuan Zhang, Douglas Eck, Chris Callison-Burch, and Nicholas Carlini. 2022. Deduplicating Training Data Makes Language Models Better. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*. ACL, 8424–8445. doi:10.18653/v1/2022.acl-long.577

[23] Jure Leskovec, Anand Rajaraman, and Jeffrey D. Ullman. 2020. *Mining of Massive Datasets* (3 ed.). Cambridge University Press.

[24] Guoliang Li, Dong Deng, Jiannan Wang, and Jianhua Feng. 2011. PASS-JOIN: A Partition-Based Method for Similarity Joins. In *Proceedings of the VLDB Endowment*, Vol. 5. 253–264. doi:10.14778/2078331.2078340

[25] Frédéric Magniez, Ashwin Nayak, Jérémie Roland, and Miklos Santha. 2011. Search via Quantum Walk. *SIAM J. Comput.* 40, 6 (2011), 1575–1600. doi:10.1137/090745854

[26] Willi Mann, Nikolaus Augsten, and Panagiotis Bouros. 2016. An Empirical Evaluation of Set Similarity Join Techniques. In *Proceedings of the VLDB Endowment*, Vol. 9. 636–647. doi:10.14778/2947618.2947620

[27] Julian McAuley, Christopher Targett, Qinfeng Shi, and Anton van den Hengel. 2015. Image-Based Recommendations on Styles and Substitutes. ACM SIGIR. doi:10.1145/2766462.2767755

[28] Ashley Montanaro. 2015. Quantum Speedup of Monte Carlo Methods. In *Proceedings of the Royal Society A*, Vol. 471. 20150301. doi:10.1098/rspa.2015.0301

[29] Ashwin Nayak and Felix Wu. 1999. The Quantum Query Complexity of Approximating the Median and Related Statistics. In *Proceedings of the 31st Annual ACM Symposium on Theory of Computing (STOC)*. ACM, 384–393. doi:10.1145/301250.301349

[30] Michael A. Nielsen and Isaac L. Chuang. 2000. *Quantum Computation and Quantum Information*. Cambridge University Press, Cambridge, UK.

[31] John Preskill. 2018. Quantum Computing in the NISQ Era and Beyond. *Quantum* 2 (2018), 79. doi:10.22331/q-2018-08-06-79

[32] Anna Primpeli and Christian Bizer. 2019. WDC Training Dataset and Gold Standard for Large-Scale Product Matching. Workshop on Data Ecosystems co-located with VLDB. http://webdatacommons.org/largescaleproductcorpus/

[33] Qiskit Development Team. 2025. *Qiskit: An Open-source Framework for Quantum Computing*. https://github.com/Qiskit/qiskit/releases/tag/2.3.0

[34] Vivek V. Shende, Stephen S. Bullock, and Igor L. Markov. 2006. Synthesis of Quantum-Logic Circuits. In *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, Vol. 25. 1000–1010. doi:10.1109/TCAD.2005.855930

[35] Ewin Tang. 2019. A Quantum-Inspired Classical Algorithm for Recommendation Systems. In *Proceedings of the 51st Annual ACM Symposium on Theory of Computing (STOC)*. ACM, 217–228. doi:10.1145/3313276.3316310

[36] Kristan Temme, Sergey Bravyi, and Jay M. Gambetta. 2017. Error Mitigation for Short-Depth Quantum Circuits. *Physical Review Letters* 119, 18 (2017), 180509. doi:10.1103/PhysRevLett.119.180509

[37] Immanuel Trummer. 2025. Cost-Based Query Optimization for Quantum Computation. In *Proceedings of the 2nd Workshop on Quantum Computing and Quantum-Inspired Technology for Data-Intensive Systems and Applications (Q-Data '25)*. doi:10.1145/3736393.3736690

[38] Immanuel Trummer and Christoph Koch. 2016. Multiple Query Optimization on the D-Wave 2X Adiabatic Quantum Computer. *Proceedings of the VLDB Endowment* 9, 9 (2016), 648–659. doi:10.14778/2947618.2947621

[39] Chuan Xiao, Wei Wang, Xuemin Lin, Jeffrey Xu Yu, and Guoren Wang. 2011. Efficient Similarity Joins for Near-Duplicate Detection. In *ACM Transactions on Database Systems*, Vol. 36. ACM, 15:1–15:41. doi:10.1145/2000824.2000825

[40] Bin Yu. 1997. Assouad, Fano, and Le Cam. In *Festschrift for Lucien Le Cam*, David Pollard, Erik Torgersen, and Grace L. Yang (Eds.). Springer, 423–435. doi:10.1007/978-1-4612-1880-7_29